

Week 1 - Day 4b: Variable Scope, Functions, and Recursion

In this exercise, you will explore various important concepts in Python programming, including variable scope, functions, and recursion.

In this set of exercises, we will explore variable scope in Python and practice writing functions. You will:

1. Understand the different types of variables in a Python program.
2. Write and test functions to perform various tasks.
3. Recursion: Understand the concept of recursion and how it can be used to solve problems.

By the end of this lab, you will have a stronger grasp of how to work with variables, write and test functions, and solve problems using recursion. Let's get started with the exercises!

Exercise 1: Types of Variables []

There are two kinds of variables in a Python program depending on where they are defined and how long they exist. Answer the following questions:

1.1 What are variables defined outside of a function called?

1.2 What are variables that only exist during the execution of a function called?

Exercise 2: Writing Functions []

Let's write some functions to practice your coding skills.

2.1 Write a function called `add_two` that adds two numbers `x` and `y`.

```
# TODO: Your code here
def add_two(x, y):
    pass

# Test your code
assert add_two(1, 10) == 11
assert add_two(2, 4) == 6

print("All tests pass. []")
```

2.2 Write a Python function called `add_elements` that given a list of numbers, it returns the sum of all numbers in the list. For example, given `[8, 2, 3, 0, 7]` the answer is 20.

```
# TODO: Your code here
def add_elements(lst):
    pass

# Test your code here
assert add_elements([8,2,3,0,7]) == 20

print("All tests pass. ☐")
```

2.3 Write a Python function called `reverse_string` that takes a string as an argument and returns the reversed string. For example, given the string "AddisCoder" the output should be "redoCsidA".

```
# TODO: Your code here
def reverse_string(s):
    pass

# Test your code here
assert reverse_string("AddisCoder") == "redoCsidA"

print("All tests pass. ☐")
```

2.4 Write a function called `miles_to_km` that takes miles as an argument and converts it into kilometers and returns the result.

Hint: 1 mile is roughly equal to 1.609 kilometers.

```
# TODO: Your code here
def miles_to_km(miles):
    pass

# Test your code here
assert miles_to_km(1) == 1.609

print("All tests pass. ☐")
```

2.5 Write a function called `get_maximum` that takes two numbers as arguments and returns the maximum of the two numbers.

```
# TODO: Your code here
def get_maximum(x, y):
    pass

# Test your code here
assert get_maximum(5, 2) == 5
assert get_maximum(-5, -10) == -5

print("All tests pass. ☐")
```

2.6 Write a function called `is_prime` that takes an integer as an argument and checks if it is a prime number or not, returning a Boolean.

```
# TODO: Your code here
def is_prime(num):
    pass

# Test your code here
assert is_prime(2)
assert not is_prime(10)

print("All tests pass. 🎉")
```

Exercise 3: Code Prediction []

What is the outcome of running the following code snippets? Try to think of the answer before running the code.

3.1.1 Predict what will be printed, then run the cell to see the actual result.

```
# Think of the answer, then run this cell to see the actual result.
def reverse(a, b):
    a, b = b, a
    return a, b

x, y = 246, 802

reverse(x, y)
```

3.1.2 Predict what will be printed, then run the cell to see the actual result.

```
# Think of the answer, then run this cell to see the actual result.
def incorrect_reverse(a, b):
    a = b
    b = a
    return a, b

x, y = 246, 802

incorrect_reverse(x, y)
```

3.1.3: Explain the difference between the **3.1.1** and **3.1.2**.

3.2.1 Predict what will be printed, then run the cell to see the actual result.

```
# Think of the answer, then run this cell to see the actual result.
def change_value(x, y):
    a = 100
    b = 200
```

```
        return x, y

a = 10
b = 20
x, y = change_value(a, b)
print(x, y)
```

3.2.2 Predict what will be printed, then run the cell to see the actual result.

```
# Think of the answer, then run this cell to see the actual result.
def change_value(x, y):
    x = 100
    y = 200

a = 10
b = 20
change_value(a, b)
print(a, b)
```

Exercise 4: Identify and Predict the Error []

4.1 One of the three print statements from the code below will not run because the code has an error.

```
a = 2
def my_func(a):
    z = a * 5
    return a, z
print(my_func(3))
print(a)
print(z)
```

Figure out which one it is and explain the error without running the code.

Now, run the code and check your answer.

```
# Think of the answer, then run this cell to see the actual result.
a = 2
def my_func(a):
    z = a * 5
    return a, z
print(my_func(3))
print(a)
print(z)
```

4.2 The global keyword is used to access and change the values of global variables inside functions. Taking note of that, try to predict the outcome of the following code.

```
var = 0

def MyFunc1():
    var = 10

def MyFunc2():
    global var
    var = 3.14

MyFunc1()
print(var)
MyFunc2()
print(var)
```

Now run the cell below and see what happens.

```
var = 0

def MyFunc1():
    var = 10

def MyFunc2():
    global var
    var = 3.14

MyFunc1()
print(var)
MyFunc2()
print(var)
```

Exercise 5: Printing Members []

Write a function `print_list` that takes a list and prints each member of the list.

```
# TODO: Your code here

# Test your code here
print_list([1, 2, 3, 4, 5])
```

Exercise 6: Calculating Average []

Write a function called `calculate_average` that takes a list of numbers as input and returns the average (mean) of the numbers. Then, test your function with your own test cases.

```
# TODO: Your code here

# Test your code
assert calculate_average([1, 2, 3, 4, 5]) == 3.0
```

```

assert calculate_average([10, 20, 30]) == 20.0
assert calculate_average([1]) == 1.0
assert calculate_average([]) == 0.0
assert calculate_average([0, 0, 0, 0]) == 0.0

print("All tests pass. ☑")

```

Exercise 7: Distinct Elements ☐

Write a Python function that takes a list and returns a new list with distinct elements of the first list. The order of the elements in the returned list does not matter.

Sample List : [1, 2, 3, 3, 3, 4, 5]

Unique List : [1, 2, 3, 4, 5]

```

# TODO: Your code here
def distinct_lst(lst):
    pass

# Test your code
distinct_lst([1, 2, 3, 3, 3, 3, 4, 5]) == [1, 2, 3, 4, 5]

print("All tests pass. ☑")

```

Exercise 8: Pangram? ☐

Write a Python function to check whether a string is a pangram or not.

Note : Pangrams are words or sentences containing every letter of the alphabet at least once.

For example: "The quick brown fox jumps over the lazy dog"

```

# TODO: Your code here
def is_pangram(s):
    pass

# Test your code
assert is_pangram('The quick brown fox jumps over the lazy dog') == True
assert is_pangram('Not a pangram') == False

print("All tests pass. ☑")

```

Exercise 9: Count Uppercase and Lowercase ☐

Write a Python function that accepts a string and prints the number of upper case letters and lower case letters.

Sample String : 'The quick Brow Fox'

Expected Output :

No. of Upper case characters : 3

No. of Lower case Characters : 12

```
# TODO: Your code here
def upper_lower(st):
    pass

# Test your code
upper_lower("The quick Brow Fox")
```